

Data Engineering Project Presentation Rubric

Criterion	Marks	Good 80–100%	Satisfactory 50–79%	Poor 0–49%
End-to-End Pipeline	30	Clear and functional pipeline from data ingestion to serving.	Pipeline is partly functional or incomplete.	Key pipeline stages are missing or unclear.
ML and Real-Time Output	30	Includes batch processing, MLOps, and a live or regularly refreshed dashboard.	Most components are included, but integration is limited.	Key components are missing or not working.
Technical Depth and Robustness	10	Uses relevant advanced tools and demonstrates robustness, such as retries, data checks, logging, deduplication, or monitoring.	Some advanced tools or robustness features are included.	Limited technical depth or little evidence of robustness.
Presentation and Explanation	30	Clear, concise, well structured, and focused on key code and design decisions.	Generally understandable, but some parts are unclear or unfocused.	Difficult to follow; key ideas or code are not clearly explained.

End-to-End Pipeline

This is a Data Engineering project. Your project should use a meaningful and reliable data source and demonstrate a clear data flow, such as ingestion, cleaning, storage, transformation, and serving.

A project that only trains a machine learning or deep learning model, without a proper data pipeline, will not receive a high score.

ML and Real-Time Output

The project should include batch data processing, model training and inference, and real-time processing.

For example:

- Show current taxi availability, weather conditions, and key trends.
- Generate predictions for the next two hours and compare earlier predictions with actual taxi availability once the actual data becomes available.
- Evaluate the prediction model every day at 8:00 AM.
- Detect unusually high or low taxi availability and trigger an alert for the affected area.

You may also present the MLOps process separately, or explain how it connects with the main data pipeline. For example, show how the model is trained, evaluated, registered, deployed for inference, and monitored. Docker may be used to package and serve the prediction API or dashboard.

Technical Depth and Robustness

Use relevant tools taught in the course, or demonstrate similar engineering ideas, such as workflow orchestration, message queues, distributed processing, data quality checks, retries, logging, monitoring, or failure handling.

A modular and well-organised system design is also encouraged.

Presentation and Explanation

Briefly explain the project problem and requirements.

Clearly describe the system architecture.

Clearly highlight one or two key technical points. Use relevant data, results, or demonstrations to support your explanation.

For example:

- Airflow DAG: Show when the workflow runs, task dependencies, and how failed tasks are retried.
- Docker Compose: Show which services are started together and how they are configured.

- Kafka Data Flow: Show how data moves from the producer to downstream processing or the dashboard.
- Dashboard Updates: Show how new data triggers refreshed charts, predictions, or alerts.

Reflection

A short reflection is **required** as part of the presentation.

Please briefly answer the following questions:

1. What was the most challenging part of your project?
2. What part of the project are you most proud of?
3. If you had more time, what would you improve or add to the system?

Project Submission

Please submit your project through:

xSite → Assessments → Dropbox → Week 02

Submission deadline: 14 September 2026, 11:59 PM

Please submit the following two files:

1. **One video file**

A recorded presentation and demonstration of your project.

2. **One PDF file**

The PDF should contain 3–5 resume-style bullet points that describe your project. These bullet points should briefly highlight the problem, data pipeline, key technologies, ML or real-time components, and main outcomes.

You do **not** need to submit your source code.